Data Structures and Introduction to Algorithmic Analysis

William T. Doan
Produced for Dr. Meghana Satpute
28 January 2025

Hi!

My name is William, and I will be the undergraduate TA for this course.

Curriculum Vitæ

June 2023: Joined Nerveli with Dr. Ranran Feng and Leon Jacobson

September 2023: Joined the CVMC Lab.

April 2024: Received a $4,500 grant from the Undergraduate Research Apprenticeship Program.

August 2024: Received the ECS Research Poster Award.

August 2024: Joined the Gibney Lab.

August 2024: Began as Undergraduate TA for CS 1436 under Dr. Brian Ricks.

September 2024: Accepted a visiting student position at the University of Arkansas at Fayetteville.

September 2024: One paper accepted into IEEE Transactions on Multimedia.

September 2024: Received a $2,250 grant from the National Science Foundation for REU under Dr. Hsu.

January 2025: Began Undergraduate TA-ship for Dr. Meghana Navanth Satpute.

Introduction (cont'd)

Research Interests.

Algorithmic design and analysis, computational complexity, wireless sensor and ad hoc networks, bioinformatics, string algorithms, distance metrics and social networks.

Website.

For more on myself, my office hours and my work, see my website at: **https://wtd-1.github.io/web/**.

Office Hours

My office is located in the ECSS Open Access Computer Lab, ECSS 2.104. I will be in the shared TA/Grader offices in ECSS 2.104A

My hours will be from:—

Tuesday, 11:30 AM to 12:30 PM;

Thursday, 11:30 AM to 12:30 PM.

Email questions to `doan[at]utdallas[at]edu`. You cannot work inside my office.

These hours are ***on-call hours***. I will commit to being available at those times, but might not be in my office. Hence, if I am not present, email me and I will be with you shortly. At times, the offices are locked and my Comet Card, for some off reason, does not work. Ergo, there is a chance we might convene elsewhere.

Extra Content

Data Structures and Algorithms is a quintessential course in the CS curriculum. In line with that, my **website** will have written content from the lecture of the day, summarizing what was gone over and potentially introducing different perspectives and ways of thinking.

I will also hold review sessions for your exams, and there may potentially be more opportunities to come. Keep abreast of any updates on eLearning or on my website.

Content Review

The Exponent Rules

Rule Name	Formula
Zero Exponent Law	$a^0 = 1$
Identity Exponent Law	$a^1 = a$
Product Law	$a^m \times a^n = a^{m+n}$
Quotient Law	$\frac{a^m}{a^n} = a^{m-n}$
Negative Exponents Law	$a^{-m} = \frac{1}{a^m}$
Power of a Power	$(a^m)^n = a^{mn}$
Power of a Product	$(ab)^m = a^m \cdot b^m$
Power of a Quotient	$\left(\frac{a}{b}\right)^m = \frac{a^m}{b^m}$

Table: The Exponent Rules

The Exponent Rules (cont'd)

From the fundamental laws of the exponents, certain corollaries arise,

$$\text{Let } x, n \in \mathbb{Z}.$$

$$x^n + x^n = 2x^n \neq x^{2n} \tag{1}$$

$$2^n + 2^n = 2^{n+1} \tag{2}$$

The Logarithm Rules

Rule Name	**Formula**
Product Rule	$\log_b(mn) = \log_b m + \log_b n$
Quotient Rule	$\log_b\left(\frac{m}{n}\right) = \log_b m - \log_b n$
Power Rule	$\log_b(m^n) = n \log_b m$
Change of Base Rule	$\log_a b = \frac{\log_c b}{\log_c a}$
Conditions: $a, b, x, y > 0$, $a, b \neq 1$, c has no restrictions.	

Table: The Logarithm Rules with Conditions

The Logarithm Rules (cont'd)

In computer science, the convention is the primary base of the logarithm is of two. Ergo, *log* and its sibling *lg* are defined as log_2 . Furthermore, there are certain theorems which will prove useful to you.

Theorem 1.1

$$logAB = logA + logB \text{ for } A, B > 0$$

Theorem 1.2

For $a, x, y \in \mathbb{R}$, $a, x, y > 0$ and $a \neq 1$,

$$x^{log_a y} = y^{log_a x}$$

On Theorem 1.1

Proof. Let $X = logA, Y = logB,$ and $Z = logAB$. Let the base of such logarithms be set to two. Then, $2^X = A, 2^Y = B,$ and $2^Z = AB$. We observe in the lattermost three equalities that combining X and Y to get XY yields $2^X \times 2^Y = AB = Z$. And, so, observing the exponential relations that exist, $X + Y = Z$, which means $logA + logB = logAB; A, B > 0$. **QED.**

Some other useful formulas, all of which can be derived in a similar manner, follow,

$$LogA/B = logA - logB$$

$$log(A^B) = BlogA$$

$$logX < X \text{ for all } X > 0$$

$$log\ 1 = 0, \quad log\ 2 = 1, \quad log\ 1,024 = 10, \quad log\ 1,048,576 = 20$$

On Theorem 1.2

Proof. Let $X = \log_a y$ and $Y = \log_a x$. By the definition of logarithms, $a^X = y$ and $a^Y = x$. Now consider the expression $x^{\log_a y}$. Substituting $x = a^Y$, we have $x^{\log_a y} = (a^Y)^{\log_a y}$. Using the property of exponents $(a^m)^n = a^{m \cdot n}$, this becomes $x^{\log_a y} = a^{Y \cdot \log_a y}$. Next, substituting $Y = \log_a x$, we get $a^{Y \cdot \log_a y} = a^{(\log_a x) \cdot (\log_a y)}$. By the property of logarithms that $a^{\log_a z} = z$, we simplify $a^{(\log_a x) \cdot (\log_a y)} = a^{\log_a (y^{\log_a x})}$. Thus, $x^{\log_a y} = y^{\log_a x}$. Since $a > 0$, $x > 0$, and $y > 0$, all steps are valid. **QED.**

Some other useful results, derived using similar steps, include,

$$\log_a(xy) = \log_a x + \log_a y$$

$$\log_a a = 1$$

On the Sum of the First n Integers

We aim to compute the sum,

$$S = \sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n.$$

To find a formula, we write the series forwards and backwards,

$$S = 1 + 2 + 3 + \cdots + n,$$

$$S = n + (n-1) + (n-2) + \cdots + 1.$$

Adding these two representations,

$$2S = (1+n) + (2+(n-1)) + (3+(n-2)) + \cdots + (n+1).$$

Each pair sums to $(n+1)$, and there are n terms in total. Thus,

$$2S = n(n+1).$$

Dividing both sides by 2,

$$S = \frac{n(n+1)}{2}.$$

Therefore, the sum of the first n integers is,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

On the Arithmetic Series

We begin with the arithmetic series,

$$S = \sum_{i=0}^{n} (a + i \cdot d) = a + (a + d) + (a + 2d) + \cdots + (a + nd).$$

To simplify,

$$S = a + a + d + a + 2d + \cdots + a + nd. \quad \rightarrow \quad S = (n+1)a + d(0 + 1 + 2 + \cdots + n).$$

Using the formula for the sum of the first n integers, we substitute thus,

$$S = (n+1)a + d \cdot \frac{n(n+1)}{2}.$$

Factoring $(n+1)$, we get,

$$S = \frac{(a + dn)(n+1)}{2}.$$

Thus, the sum of the arithmetic series is,

$$\sum_{i=0}^{n} (a + i \cdot d) = \frac{(a + dn)(n+1)}{2}.$$

On the Arithmetic Series (cont'd)

In particular, for the sum of the first n integers, we have:

$$\sum_{i=0}^{n} i = \frac{n(n+1)}{2} = \Theta(n).$$

This shows that the sum grows linearly with n. More generally, for powers of integers, we can show,

$$\sum_{i=0}^{n} i^c = \Theta(n^{c+1}) \quad \text{for any constant } c > -1.$$

This result indicates the sum of powers grows asymptotically at the rate of n^{c+1}. The exponent $c+1$ determines how fast the sum increases as n grows.

On the Arithmetic Series (cont'd)

If $c = -1$ (and we start with $i = 1$), the sum becomes the nth Harmonic number,

$$H_n := \sum_{i=1}^{n} 1/i = \Theta(\log n).$$

This shows that the Harmonic number grows logarithmically with n, much slower than polynomial growth. If $c < -1$, the sum converges to a constant,

$$\Theta(1).$$

Here, terms decay so quickly that the total sum stabilizes and does not grow with n.

On the Geometric Series

These are less involved than their arithmetic cousins since they can be derived in much the same way. The easiest formulas for remember are,

$$\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$$

as well as the companion,

$$\sum_{i=0}^{n} A^i = \frac{A^{n+1} - 1}{A - 1}$$

In the latter, if $0 < A < 1$, then,

$$\sum_{i=0}^{n} A^i \leq \frac{1}{A - 1}$$

as n tends to ∞.

On Modular Arithmetic

Recall the definition of the modulo. Some number A, then, is congruent to another number B modulo n, written mathematically $A \equiv B \ (mod\ n)$, if and only if n divides $A - B$. That means there exists some constant k such that,

$$A - B = k \ \times \ n$$

Suffice to say, $A \equiv B \ (mod\ n)$ means that A and B have the same remainder when both are divided by the same number n. Therefore, we can say $81 \equiv 61 \equiv 1 \ (mod\ 10)$.

On Modular Arithmetic (cont'd)

When we contemplate modular arithmetic, the n value we toy with is usually prime. From that idea, three theorems blossom,

Thm. 2.1. If n is prime, then $ab \equiv 0 \ (mod\ n)$ if and only if $a \equiv 0 \ (mod\ n)$ **OR** $b \equiv 0 \ (mod\ n)$. Intuitively, the product of two numbers, divided by n, means n itself divides one of the constituents of that product.

Thm. 2.2. If n is prime, the equation $ax \equiv 1 \ (mod\ n)$ has a unique solution $(mod\ n)$ for all a values such that $0 < a < n$ called the *multiplicative inverse*. $ax \equiv 1 \ (mod\ n)$ is analogous to $x = 1/a$ in modular arithmetic. This is a result guaranteed by *Bezout's Identity*.

Thm. 2.3. If n is prime, the equation $x^2 \equiv a \ (mod\ n)$ either has two solutions $(mod\ n)$ for all $0 < a < n$ or none at all. I.e., a is either a quadratic residue modulo n or it is not.

Proofs by Induction

Proofs by induction seek to prove some idea is true for all cases of n. For instance, let us prove $3^n > n^3$ for $n \geq 4$.

Proof. For a base case of $n = 4$,

$$3^4 > 4^3 \rightarrow 81 > 64, \text{ which is true.}$$

Before we proceed with our proof by induction, we must define an inductive hypothesis. The reason for that is we need to assume the proposition holds for a particular value. Recall the definition–to prove something is true for all cases, we must begin somewhere and prove the next step up.

Inductive Hypothesis. $3^n > n^3$.

Proofs by Induction (cont'd)

To prove the next step up, which is $3^{n+1} > (n+1)^3$,

$$3 \times 3^n > (n+1)(n+1)(n+1)$$

Notice the left hand side—it is thrice the inductive hypothesis. So, by substitution,

$$3 \times (3^n > n^3) > (n+1)(n+1)(n+1)$$

$$\rightarrow (3 \times 3^n > 3n^3) > (n+1)(n+1)(n+1)$$

We realize, now, that in the inner inequality on the left hand side, that $3n^3$ is the lower bound. So, it is the term we care about,

$$3n^3 > (n+1)(n+1)(n+1)$$

Proofs by Induction (cont'd)

Now, we can simplify,

$$3n^3 > (n+1)(n+1)(n+1)$$

$$\rightarrow 3n^3 > n^3 + 3n^2 + 3n + 1$$

$$\rightarrow 2n^3 > 3n^2 + 3n + 1$$

This result is true from the inductive hypothesis— for the case of $n = 4$, we will find $128 > 61$, which is true. For all successive numbers, from four and for $n \in \mathbb{Z}$, $2n^3$ will always be asymptotically dominant as it is the term of highest order.

Based on that and all our work,

$$3 \times 3^n > 3n^2 + 3n + 1 \rightarrow 3 \times 3^n > (n+1)^3$$

$$\rightarrow 3^{n+1} > (n+1)^3 \rightarrow 3^n > n^3. \textbf{ QED.}$$

Proofs by Contradiction

Proofs by Contradiction exist for when proofs by induction or direct proofs would take too long to establish continuity and reach a conclusion. We begin by assuming the proposition is true, and proceed in the fashion of a direct proof, or even an inductive one, until we are proven otherwise by arriving at a nonsensical conclusion.

Many of you should be familiar with a proof by contradiction from Lab 11.

There is another proof, the proof by counterexample, which is a cousin of the proof by contradiction. For any universally quantified statement, you can attempt to disprove it by producing a singluar counterexample which proves such statement cannot be justified for all cases as it claims.

Recursion

Suppose a sequence, $f(0) = 0$ and $f(n) = 2f(n-1) + n^2$.

$$f(1) = 1 \quad f(2) = 6 \quad f(3) = 21 \quad f(4) = 58$$

We observe that we were able to derive each successive value for n given the previous case. This means the function f is recursive. In programming, this means your functions is calling other functions to use what the latter returns.

Recursion (cont'd)

Recursion comes with a set of properties you must observe,

1.) There must always exist some number of base cases—they are those values given without the need to derive them using recursion.

2.) For each recursive case, the recursive call must always be to a case that makes progress toward the base case. I.e., you must be able to backtrack.

3.) All recursive calls must work (can be assumed and taken as a given).

4.) Work can never be duplicated by solving the same instance of a problem in separate recursive calls.

Introduction to Asymptotic Notation

To study algorithms effectively, we must answer a key question: how long does an algorithm take as input size n grows? Instead of counting the exact number of operations (a tedious task), we use a more practical tool—**asymptotic notation**.

Asymptotic notation allows us to describe an algorithm's growth rate as n increases while ignoring constant factors and lower-order terms. This allows us to compare algorithms based on their *scalability*, not their raw instruction count.

Big-O Notation (O)

Big-O notation gives an upper bound on the growth rate of a function. It tells us that the function grows no faster than a given comparison function, $g(n)$, up to a constant factor.

$$f(n) \in O(g(n)) \quad \text{if} \quad f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0,$$

where $c > 0$ and $n_0 > 0$ are constants.

E.g., the sum of the first n integers is,

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

As n grows, the leading term is $n^2/2$, so,

$$\sum_{i=1}^{n} i \in O(n^2).$$

But this bound is loose. The actual growth is $\Theta(n)$.

Tight Bounds with Big-Θ

While O-notation gives a loose upper bound, **Big**-Θ provides a tight bound, meaning $f(n)$ grows exactly as fast as $g(n)$, up to constant factors.

$$f(n) \in \Theta(g(n)) \quad \text{if} \quad c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0,$$

where $c_1, c_2 > 0$ are constants.

E.g., for the sum $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$,

$$f(n) \in \Theta(n).$$

This means the sum grows linearly with n, as the $n^2/2$ term dominates the lower-order constant terms. This helps us communicate both an algorithm's best and worst behavior in one clean statement.

Lower Bounds with Big-Ω

Big-Ω gives a lower bound on the growth rate of a function. It tells us that $f(n)$ grows at least as fast as $g(n)$, up to constant factors.

$$f(n) \in \Omega(g(n)) \quad \text{if} \quad f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0,$$

where $c > 0$ and $n_0 > 0$ are constants.

E.g., if an algorithm requires at least n operations to process an input of size n, then,

$$f(n) \in \Omega(n).$$

This tells us the algorithm cannot run faster than linear time in the worst case.

Strict Bounds: Little-o and Little-ω

Little-o and **Little**-ω are stricter notations than O and Ω.

Little-o is a strict upper bound. $f(n) \in o(g(n))$ if:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

Little-ω is a strict lower bound. $f(n) \in \omega(g(n))$ if:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty.$$

E.g., the function $n \in o(n^2)$ because as n grows, $n/n^2 \to 0$. Conversely, $n^2 \in \omega(n)$ because $n^2/n \to \infty$.

These bounds help us express very tight growth relationships.

Remarks on Asymptotic Notation

Asymptotic notation simplifies our lives,

- $\rightarrow$ It allows us to focus on dominant terms and ignore constants.
- $\rightarrow$ It makes algorithm comparisons straightforward.
- $\rightarrow$ It provides a universal language for algorithm efficiency.

For example, if one algorithm has a running time of n^2 and another has a running time of $2n^3$, we can immediately say,

$$n^2 \in o(n^3),$$

and the n^2-algorithm is asymptotically faster.

This clarity is essential when analyzing data structures, designing algorithms, or tackling competitive programming problems.

Infimum (inf) and Supremum (sup)

To better understand asymptotic notation, we use mathematical tools like the **infimum** and **supremum**:

> inf: The greatest lower bound of a set, meaning no element in the set is smaller, though elements outside the set might be.
>
> sup: The least upper bound of a set, meaning no element in the set is greater, though elements outside the set might be.

E.g., for the set $S = \{1/n : n \in \mathbb{N}, n \geq 1\}$:

$$\inf(S) = 0, \quad \sup(S) = 1.$$

These concepts refine our ability to describe bounds, bridging O, Ω, and Θ.

Using inf and sup in Asymptotic Analysis

We analyze the growth of functions as $n \to \infty$. Using inf and sup, we define asymptotic bounds:

Big-O: $f(n) \in O(g(n))$ if

$$\sup_{n \geq n_0} \frac{f(n)}{g(n)} < \infty.$$

Big-Ω: $f(n) \in \Omega(g(n))$ if

$$\inf_{n \geq n_0} \frac{f(n)}{g(n)} > 0.$$

Big-Θ: $f(n) \in \Theta(g(n))$ if

$$0 < \inf_{n \geq n_0} \frac{f(n)}{g(n)} \leq \sup_{n \geq n_0} \frac{f(n)}{g(n)} < \infty.$$

These formal definitions provide rigorous tools for bounding growth.

Why inf and sup Matter

Infimum and supremum are invaluable for asymptotic analysis:

Clarity. They give precise mathematical conditions for bounding functions.

Tightness. They help distinguish between loose bounds (O) and tight bounds (Θ).

Universality. They apply to irregularly growing functions as well.

E.g., for $f(n) = 3n^2 + 5n + 2$ and $g(n) = n^2$:

$$\sup_{n\to\infty} \frac{f(n)}{g(n)} = 3, \quad \inf_{n\to\infty} \frac{f(n)}{g(n)} = 3.$$

Thus, $f(n) \in \Theta(g(n))$, as both bounds match. These tools deepen our understanding of algorithmic efficiency.